# Guía para presentar el código · Dashboard 360 de Olist

Esta guía es solo para estudiar. No se sube al repositorio. Está basada en el código que quedó terminado: `pipeline.py`, `transformar.py`, `dashboard.py` y `test_pipeline.py`.

---

## 1. Quién presenta qué

| Parte | Quién | Archivos |
|---|---|---|
| Presentación en PowerPoint | Roberto abre (portada y empresa Olist) y sigue Favio; ver el cuadro de la sección 5 | `Dashboard360_presentacion.pptx` |
| Presentación del código | **Favio**: el flujo de datos | `pipeline.py` y `transformar.py` |
| Presentación del código | **Roberto**: el dashboard y las pruebas | `dashboard.py` y `test_pipeline.py` |
| Demostración en vivo | Cualquiera de los dos | la terminal |

Los dos tienen que saber todo el proyecto. El profesor puede preguntarle a cualquiera sobre cualquier archivo.

**Idea central que repiten los dos:** un solo comando (`python pipeline.py`) lee 6 tablas de Olist y una API, las valida, las transforma, las guarda y abre un dashboard. Responde una pregunta: *¿cómo va el negocio y dónde conviene actuar primero?*

---

## 2. FAVIO · `pipeline.py` y `transformar.py`

### Orden para mostrarlo

**A. Arriba de `pipeline.py` (líneas 1 a 35): el docstring** Aquí está el problema de negocio, la pregunta, para quién, por qué importa y la decisión que permite. Se lee en 20 segundos. Termina con los 3 pasos, que son como una cocina: extraer, transformar y cargar.

**B. Parámetros (líneas 55 a 65)**

- `load_dotenv()` lee el archivo `.env`, que está fuera del código y no se sube a git.
- `URL_DOLAR = os.getenv("URL_DOLAR", "...")`: la dirección de la API sale del `.env`. Si no existe, usa la de por defecto.
- Aquí están los valores que se pueden cambiar sin tocar la lógica: la carpeta de datos, la de salida y el valor de respaldo del dólar.
- Frase: *"No hay claves ni contraseñas en el código. La API es pública y de todas formas su dirección va en el .env."*

**C.** `validar_tabla` **(línea 71)** Revisa dos cosas: que la tabla no esté vacía y que tenga las columnas que vamos a usar. Si algo falla, lanza un `ValueError` con un mensaje claro. Se prueba con 3 tests.

**D.** `extraer_olist` **(línea 80)** Lee los 6 CSV: pedidos, ítems, reseñas, clientes, productos y la traducción de categorías. Después valida cada uno. Frase: *"Los otros 3 CSV (pagos, geolocalización y vendedores) no responden la pregunta, así que no los usamos."*

**E.** `extraer_dolar` **(línea 113)** Llama a la API `mindicador.cl` con `requests.get(..., timeout=10)`. Si la API no responde, atrapa el error, deja una advertencia en el log y sigue con el valor de respaldo (180 pesos por real). Frase: *"Es el plan B: el pipeline no se cae porque una API externa falle."*

**F.** `cargar` **(línea 136)** Guarda cada tabla en SQLite y en un CSV con `to_sql(..., if_exists="replace")`. Frase clave: *"Con replace, si lo ejecutas dos veces la tabla se reescribe y queda igual. No se duplican filas. Por eso las dos corridas dan el mismo resultado."*

**G.** `main()` **(línea 175)** Es donde se ve el flujo completo:

1. Configura el `logging` (a la pantalla y al archivo `salida/pipeline.log`).
2. `try/except`: si falta un CSV (`FileNotFoundError`) o una tabla no es válida (`ValueError`), deja un error claro en el log y termina sin romperse.
3. Llama a las 5 funciones de transformación.
4. Guarda con `cargar`, crea el dashboard y muestra el resumen para la gerencia.

### `transformar.py`: muestra solo dos o tres funciones

`preparar_pedidos` **(línea 38)** · limpieza

- Se queda solo con los pedidos entregados (`order_status == "delivered"`).
- Convierte las fechas de texto a fecha y calcula `dias_atraso` = fecha real − fecha prometida. Negativo significa que llegó antes.
- Si no hay fecha de entrega, queda vacío (NaN). No se inventa un dato.

`satisfaccion_por_atraso` **(línea 157)** · la que responde la pregunta

1. Promedia las reseñas por pedido (hay pedidos con más de una reseña).
2. `merge` de pedidos con reseñas por `order_id`.
3. Clasifica cada pedido en un tramo con `clasificar_atraso` (a tiempo, 1 a 3 días, 4 a 10, más de 10).
4. `groupby` por tramo: cuántos pedidos y qué nota promedio tienen. - Resultado: la nota baja de **4,29** (a tiempo) a **1,71** (más de 10 días de atraso).

`entregas_por_estado` **(línea 189)** · si te quedan minutos `merge` de pedidos con clientes por `customer_id`, `groupby` por estado y porcentaje a tiempo. Salen **AL, MA y SE** como los peores.

Todas las funciones tienen en el docstring dos líneas: **Negocio** (para qué sirve) y **Tablas** (qué se une con qué).

### Los números que tienes que saber de memoria

- 99.441 pedidos; 96.478 entregados; 2.963 no entregados (se excluyen).
- Ventas: 13.221.498,11 BRL; ticket promedio 137,04; satisfacción 4,16 de 5; entregas a tiempo 93,23 %.

- Tramos de atraso y nota: 4,29 · 3,29 · 1,97 · 1,71.
- Peores estados: AL 78,59 %, MA 82,57 %, SE 84,78 %.
- Categoría que más vende: `health_beauty`.

### Preguntas probables y cómo responder

**¿Qué pasa si falla la API del dólar?** Se atrapa el error en `extraer_dolar`, se avisa en el log y se usa el valor de respaldo. El pipeline sigue.

**¿Qué pasa si falta un CSV?** `main()` atrapa el `FileNotFoundError` y deja un mensaje claro en el log: que no encontró los CSV y que revise si corrió el pipeline desde la carpeta del proyecto.

**¿Por qué dos ejecuciones dan el mismo resultado?** Porque se guarda con `if_exists="replace"` en vez de agregar filas. Hay una prueba que lo comprueba (`test_cargar_dos_veces_no_duplica_filas`).

**¿Para qué sirve el `.env`?** Para sacar la configuración del código. El `.env` no se sube a git (está en el `.gitignore`) y se deja un `.env.example` para que cualquiera lo copie.

**¿El resultado puede cambiar entre ejecuciones?** No: dos ejecuciones el mismo día dan exactamente lo mismo. La única cifra que depende de algo externo es `ventas_totales_clp`, porque usa el dólar que entrega la API, y ese valor se actualiza una vez al día. Si se ejecutara otro día, solo esa cifra en pesos podría cambiar. Las cifras en reales no cambian nunca. Además el factor de 0,19 dólares por real es fijo, así que el valor en pesos es aproximado.

**¿Por qué excluyen los pedidos no entregados?** Porque solo lo entregado cuenta como venta y puede tener una reseña confiable.

**¿Qué es un merge y qué es un groupby?** El `merge` une dos tablas por una columna en común (por ejemplo `order_id`). El `groupby` agrupa filas y las resume (contar o promediar). Se registra en el log cuántas filas hay antes y después de cada merge.

**¿El atraso causa la nota baja?** Solo podemos decir que van juntos: es una asociación, no prueba la causa.

**¿Usaron IA?** Sí, Claude Code como ayudante para escribir y revisar el código y los tests. Pero corrimos todo con los datos reales, simplificamos el proyecto a lo visto en clase y lo revisamos contra lo que pide el profesor.

---

## 3. ROBERTO · `dashboard.py` y `test_pipeline.py`

### `dashboard.py`: orden para mostrarlo

**A. Arriba del archivo (líneas 1 a 10)** El dashboard es una página web (`salida/dashboard.html`) que se abre con doble clic. No necesita servidor ni internet. La crea `pipeline.py` al terminar.

**B. `crear_dashboard` (línea 84)** Arma una pantalla con 4 tarjetas arriba y 3 gráficos abajo, con Plotly:

- **Tarjetas** (función `tarjetas`, línea 33): ventas 13,2 M BRL, ticket 137 BRL, satisfacción 4,16 de 5, entregas a tiempo 93,2 %.
- **Gráfico 1:** categorías que más venden. Las 3 primeras van en azul y el resto en gris.
- **Gráfico 2:** la nota según los días de atraso. Cae de 4,29 a 1,71.

- **Gráfico 3:** los estados con peor puntualidad. Los 3 peores van en rojo.

**C. La parte interactiva**

- Al pasar el mouse por una barra se ve el detalle: ventas y unidades, o pedidos y atrasos.
- El botón desplegable (arriba a la derecha del gráfico 3) elige mostrar los 5 peores estados, los 10 peores o todos.
- El código del botón está en `crear_dashboard`, alrededor de las líneas 121 a 131.

**D. Colores con intención** Gris para lo normal y color para lo que importa. Así el gerente mira primero lo que debe atender.

**E. `texto_decision` (línea 70)** El recuadro de abajo se calcula con los resultados: nombra los 3 peores estados y las notas extremas. Si cambian los datos, el texto cambia solo.

Frase para cerrar: *"El dashboard solo dibuja lo que ya calculó el pipeline. Si fallara, los resultados igual quedan guardados en SQLite y en los CSV."*

### `test_pipeline.py`: orden para mostrarlo

1. Explica la idea (docstring de arriba): cada prueba usa tablas pequeñas creadas en memoria, con un resultado que se puede calcular a mano.
2. Ejecuta `pytest -v` y muestra **22 passed**.
3. Muestra una prueba. Buenos ejemplos: - `test_preparar_pedidos_calcula_dias_de_atraso` (línea 77): el pedido A llega 2 días tarde y el B 2 días antes, así que espera `[2, -2]`. - `test_indicadores_promedia_resenas_y_excluye_sin_resena` (línea 108): un pedido con reseñas 5 y 3 promedia 4; uno con 2; uno sin reseña no cuenta. El promedio final es 3,0.
4. Cuántas pruebas hay por función: 3 para `validar_tabla`, `preparar_pedidos`, `calcular_indicadores`, `top_categorias`, `clasificar_atraso`, `satisfaccion_por_atraso` y `entregas_por_estado`; más 1 para `cargar`. Son 22 en total.

### Preguntas probables y cómo responder

**¿Por qué usaron Plotly si no se vio en clase?** Es solo la forma de mostrar los resultados. Está aislado en su propio archivo y los resultados se guardan en SQLite y CSV antes de dibujar el dashboard, así que esos datos ya quedan a salvo aunque el dibujo fallara.

**¿Cómo saben que los resultados son correctos?** Con las pruebas de pytest (tablas pequeñas con resultado calculable a mano) y porque dos ejecuciones dan el mismo resultado.

**¿Qué decisión permite el dashboard?** Atacar primero los atrasos de más de 10 días (la nota baja de 4,29 a 1,71) y revisar primero la logística en AL, MA y SE.

**¿Qué mejorarían?** Programar el pipeline cada semana y conectar Power BI a `dashboard360.db`. Agregar los costos para medir rentabilidad.

**¿Qué limitación tiene?** Los datos son de 2016 a 2018, no hay costos y la relación entre atraso y nota es una asociación.

---

## 4. La demostración en vivo (la hace cualquiera de los dos)

Desde Git Bash, en la carpeta `Dashboard360`, con el entorno activo (`source .venv/Scripts/activate`):

```
python pipeline.py     # 1ª vez: se ve el log y se abre el dashboard
python pipeline.py     # 2ª vez: exactamente los mismos números
pytest -v              # 22 passed
```

Qué se ve y qué decir:

- En el log: `Olist leído: 99441 pedidos...`, los merge con sus filas, `Guardada la tabla...` y `Dashboard 360: terminado`.
- En el resumen: pedidos entregados 96.478, ventas, nota por tramo y los 5 estados peores.
- En el navegador: el dashboard. Mover el mouse y usar el botón de estados.

**Plan B** si algo falla: abrir `salida/dashboard.html` con doble clic y mostrar las capturas guardadas. Ensayar todo el día antes, con los CSV en `data/raw/`.

---

## 5. La presentación en PowerPoint: quién habla en cada diapositiva

| N.º | Diapositiva | Habla | Tiempo |
|---|---|---|---|
| 1 | Portada | Roberto | 0,5 min |
| 2 | La empresa: Olist | Roberto | 1 min |
| 3 | El problema de negocio | Favio | 1,5 min |
| 4 | Los datos | Favio | 1,5 min |
| 5 | Arquitectura del pipeline | Favio | 1,5 min |
| 6 | Demo en vivo | Cualquiera de los dos | 3 min |
| 7 | Resultados | Roberto | 2 min |
| 8 | Calidad y buenas prácticas | Roberto | 1 min |
| 9 | Valor de negocio | Roberto | 1,5 min |
| 10 | Aprendizajes e IA | Favio | 1 min |
| 11 | Preguntas | los dos | 5 min |

Total: 14,5 minutos más las preguntas. El texto de lo que dice cada uno está en las **notas del orador** de cada diapositiva.

Para pasar la palabra: Roberto cierra su introducción con *"con esos datos armamos una vista única para el gerente comercial; Favio les cuenta el problema"*.

---

## 6. Qué significan AL, MA, SE... (siglas de los estados de Brasil)

Son los **estados de Brasil** donde viven los clientes de Olist. En el código salen de la columna `customer_state` de la tabla de clientes. El dashboard tiene 27 porque son todos los estados de Brasil (26 estados más el Distrito Federal): con el botón "Todos" del gráfico 3 se ven los 27.

| Sigla | Estado | Sigla | Estado | Sigla | Estado |
|---|---|---|---|---|---|
| **AC** | Acre | **MA** | Maranhão | **RJ** | Río de Janeiro |
| **AL** | Alagoas | **MG** | Minas Gerais | **RN** | Río Grande do Norte |
| **AM** | Amazonas | **MS** | Mato Grosso do Sul | **RO** | Rondônia |
| **AP** | Amapá | **MT** | Mato Grosso | **RR** | Roraima |
| **BA** | Bahía | **PA** | Pará | **RS** | Río Grande do Sul |
| **CE** | Ceará | **PB** | Paraíba | **SC** | Santa Catarina |
| **DF** | Distrito Federal (Brasilia) | **PE** | Pernambuco | **SE** | Sergipe |
| **ES** | Espírito Santo | **PI** | Piauí | **SP** | São Paulo |
| **GO** | Goiás | **PR** | Paraná | **TO** | Tocantins |

**Los tres que salen en rojo (los peores en puntualidad):**

| Estado | Pedidos | Atrasados | Entregas a tiempo |
|---|---|---|---|
| **AL** · Alagoas | 397 | 85 | 78,59 % |
| **MA** · Maranhão | 717 | 125 | 82,57 % |
| **SE** · Sergipe | 335 | 51 | 84,78 % |

Los que aparecen después en el gráfico: PI (Piauí) 86,13 %, CE (Ceará) 86,24 %, RR (Roraima), BA (Bahía), RJ (Río de Janeiro), PA (Pará) y ES (Espírito Santo).

**Cómo decirlo en la presentación:** *"Estos son estados del nordeste de Brasil. Los tres peores en puntualidad son Alagoas, Maranhão y Sergipe, así que ahí conviene revisar primero la logística."*

**Si el profesor pregunta por la muestra:** AL y SE tienen pocos pedidos (397 y 335) comparados con otros estados. El resultado es una pista para revisar, no una prueba definitiva.